OSU Satellite TV for MITRE eCTF 2025

Contents

1. Team

The team is comprised of the following people from the Ohio State State University:

Student	Advisor
Christopher Begines	Julia Armstrong
Mark Bundschuh	Dr. Felix Engelmann
William Comer	
Mason Erwine	
Aditya Gupta	
Ryan Jackman	
Cadan Keller	
Jack Leverett	
Christopher McDevitt	
Maxwell McGee	
Jacob Mcquade	
Diego Noria	
Divij Sasidhar	
Joshua Sims	
Eva Smith	
Matthew Weikel	
Zihao Zhang	

2. Introduction

This document describes OSU's implementation of a secure satellite TV system as specified by MITRE's eCTF 2025. The Satellite TV system features a simulated satellite sending ASCII art to subscribed hardware Decoder devices, implementing Functional Requirements 2.1. The decoder devices are MAX78000FTHR microcontrollers by Analog Devices. The system is secured according to the Security Requirements 2.2, and designed to defend against attacks as outlined in Attack Scenarios 2.3, which revolve around erroneously decoding frames sent by a satellite.

2.1. Functional Requirements

The two main components are the Encoder and Decoder. The encoder receives frames of up to 64 bytes, and converts them into our custom format before sending it over the satellite link. Then, each decoder receives the frame, and must decode the message into the original 64 bytes.

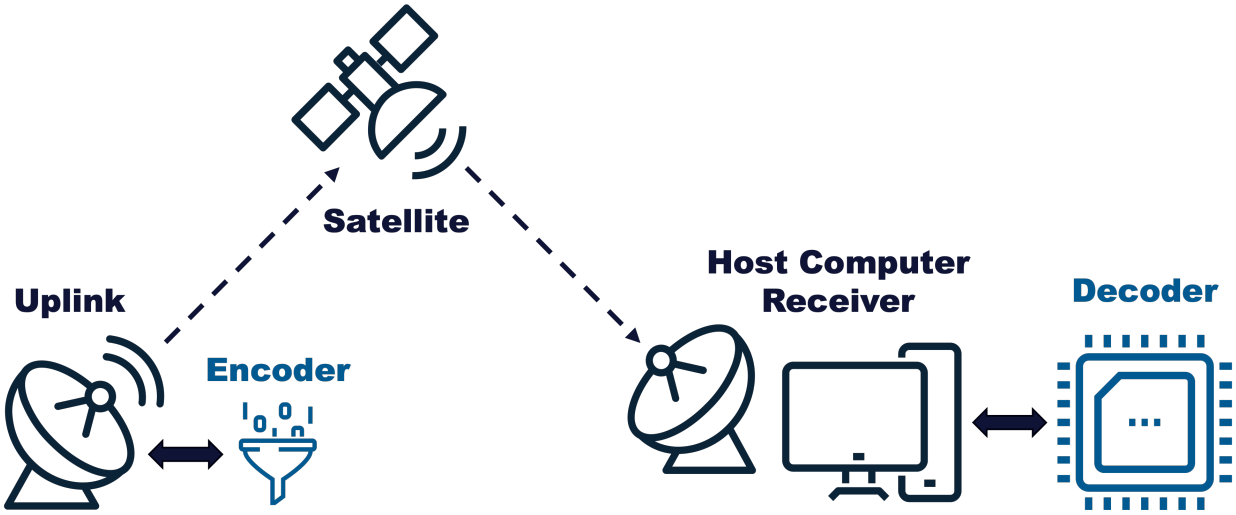

Figure 1: Full system with blue highlights indicating components which need to be implemented. [1]

2.1.1. Functional Requirement 1: List Channels

"The Decoder must be able to list the channel numbers that the Decoder has a valid subscription for." [1]

2.1.2. Functional Requirement 2: Update Subscription

"The Decoder must be able to update it's channel subscriptions with a valid update package. If a subscription update for a channel is received for a channel that already has a subscription, the Decoder must only use the latest subscription update." [1]

2.1.3. Functional Requirement 3: Decode Frame

"The Decoder must properly decode TV frame data from any channel that has a valid and active subscription installed or for any emergency broadcast frames." [1]

2.2. Security Requirements

2.2.1. Security Requirement 1: Active Subscription

"An attacker should not be able to decode TV frames without a Decoder that has a valid, active subscription to that channel." [1]

See Section 4.1 on how the design protects against the security requirement.

2.2.2. Security Requirement 2: Authentication

"The Decoder should only decode valid TV frames generated by the Satellite System the Decoder was provisioned for." [1]

See Section 4.3 on how the design protects against the security requirement.

2.2.3. Security Requirement 3: Increasing Timestamps

"The Decoder should only decode frames with strictly monotonically increasing timestamps." [1]

See Section 4.4 on how the design protects against the security requirement.

2.3. Attack Scenarios

We consider 5 different scenarios where the attackers have access to certain information that the system is designed to protect against.

2.3.1. Attack Scenario 1: Expired Subscription

An attacker has physical access to a decoder with a valid subscription to a channel. The timestamp of the current frames sent by the encoder is past the ending timestamp of the attacker's subscription.

See Section 4.1 on how the design protects against the attack scenario.

2.3.2. Attack Scenario 2: Pirated Subscription

An attacker has physical access to a decoder, and a valid subscription to a channel for a different decoder which they do not have physical access to.

See Section 4.2 on how the design protects against the attack scenario.

2.3.3. Attack Scenario 3: No Subscription

An attacker has physical access to a decoder, but has no subscriptions.

See Section 4.1 on how the design protects against the attack scenario.

2.3.4. Attack Scenario 4: Recording Playback

An attacker has physical access to a decoder, and has been recording the raw data sent by the satellite for some time. Then, they get a valid subscription for the channel. In other words, the attacker should not be able to read past frames given an active subscription.

See Section 4.1 on how the design protects against the attack scenario.

2.3.5. Attack Scenario 5: Pesky Neighbor

An attacker has physical access to a decoder, and spoofs the satellite signal to a neighbor's decoder which has a valid subscription. The attacker does not have physical access to the neighbor's decoder.

See Section 4.3 on how the design protects against the attack scenario.

3. Deployment

3.1. Secrets

The following secrets are generated for a deployment. Once generated, they are global and read only. The deployment secrets are available to the encoder. Some of these secrets will be shared with the Decoders in various forms as described by Section 3.2.

- $S_1, ..., S_8$
  - Up to 8 32 byte randomly generated seeds, one per encrypted channel
- $S_{\text{subscription}}$
  - Randomly generated 64 byte subscription salt
- $K_{\text{secret}}, K_{\text{public}}$
  - An Ed25519 asymmetric key pair
- K_{metadata}
  - Randomly generated 32 byte shared symmetric key

3.2. Decoder

The firmware must be built for each separate decoder device. When building, specify the `DEVICE_ID` environment variable as a 4 byte hexadecimal number starting with `0x`, denoted as D_{id}.

At build time, the decoder gathers the following secrets and stores them within the encrypted firmware:

- K_{metadata}
  - Shared 32 byte symmetric key with the Encoder which is used to encrypt the metadata (timestamp, channel id), and
- K_{public}
  - Shared public key with the Encoder
- $K_{\text{subscription}}$
  - Derived as $\text{blake3}\left(S_{\text{subscription}} + D_{\text{id}}\right)$, and is the symmetric key used to decrypt subscriptions
- $C_1, ..., C_8$
  - List of up to 8 Channel IDs which the decoder was provisioned with
- K_{flash}
  - Randomly generated 32 byte symmetric used to encrypt data before writing it to the flash storage

4. System Design

4.1. Encode and Decode

Field Name	Size	Relative Size
Signature	64 bytes	
Channel ID	2 byte	
Timestamp	8 bytes	
Message	up to 64 bytes	

Table 1: Structure of a decode packet

The plaintext contents *Channel ID*, *Timestamp*, and *Message* are signed with K_{private} to form the *Signature* which is prepended to the data. Then, those three fields are encrypted with the symmetric K_{metadata} key using the first 8 bytes of *Signature* as a nonce, which forms the final decode packet.

Each frame has an integer timestamp $t \in [0, 2^{64} - 1]$. Each timestamp has a unique symmetric key k_t which the encoder will use to encrypt the *Message*, and the decoder will use to decrypt the *Message*, both using Salsa20. Note that this key is on top of k_{metadata} which is used to encrypt the entire packet payload. The keys are derived with a *binary hash key derivation tree*. Let H be a cryptographic hash function. We have chosen $H = \text{blake3}$. Let L and R be salted hash functions $L(x) = H(x + \text{“L”})$ and $R(x) = H(x + \text{“R”})$. The encoder uses secret $S_{\text{channel\_id}}$ for the channel id currently being broadcasted, and generates a root hash $h = H(S)$. Then, two hashes $h_0 = L(h)$ and $h_1 = R(h)$ are derived. Then $h_{00} = L(h_0)$, $h_{01} = R(h_0)$, $h_{10} = L(h_1)$, and $h_{11} = R(h_1)$, and so on. This process continues to form a tree of height 64. The leaves of the tree form the keys k_t, one for every timestamp. Figure 2 shows a hash key derivation tree for $t \in [0, 7]$ and has 8 leaves. Note that the full sized tree has 2^{64} leaves, the leaves are computed on demand for particular timestamps since it would be computationally infeasible to precompute all keys for such a large tree.

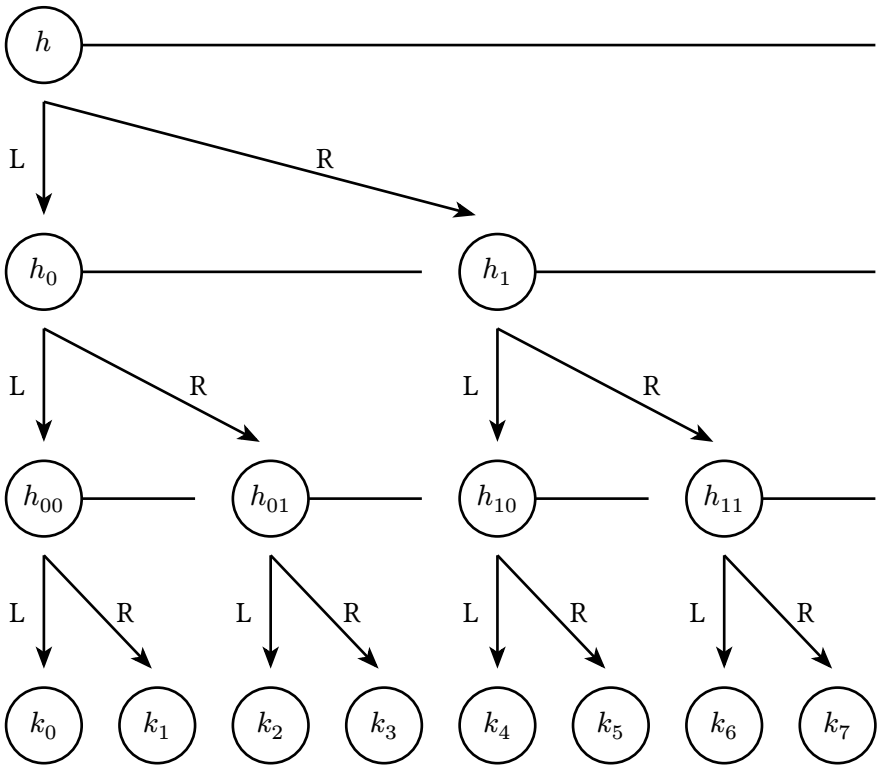

Figure 2: A hash key derivation tree from $t = 0$ to $t = 7$

To send the keys to the decoder, they are compressed into a subscription file. The compression is done by finding the largest power of two intervals which span the entire interval of timestamps $t \in [\text{start}, \text{end}]$. For example, all keys in $[1, 5]$ can be derived by $h_{001} \Rightarrow \{k_1\}$, $h_{01} \Rightarrow \{k_2, k_3\}$, and $h_{10} \Rightarrow \{k_4, k_5\}$, so only h_{001}, h_{01}, and h_{10} are needed to form a subscription. The worst case interval is $[1, 2^{64} - 2]$ which requires 126 subtree roots.

This aspect of the design secures the system in the following ways:

- **SR1**: Active Subscription
  - The decoder is never given more keys than exactly those which correspond to the timestamps in the subscription it was given. This means it is cryptographically impossible to decrypt frames outside of the subscription interval.
- **AS1**: Expired Subscription
  - The decoder is never given keys for the frames past the end of the subscription interval, so it is cryptographically impossible to decrypt frame data past the end of its subscription.
- **AS3**: No Subscription
  - Attackers without any subscription have no keys for any frames, so they are unable to decode any frames at all.
- **AS4**: Recording Playback
  - The decoder is never given keys for the frames outside of the subscription interval, so it is not possible to decrypt past frames.

4.2. Subscription

Field Name	Size	Relative Size
Signature	64 bytes	
Start Timestamp	8 bytes	
End Timestamp	8 bytes	
Channel ID	2 bytes	
Root Hashes	$\{24n \text{ bytes} \mid n \leq 126\}$	$\ldots n$ times

Table 2: Structure of a subscribe packet

The plaintext contents *Start Timestamp*, *End Timestamp*, *Channel ID*, and *Root Hashes* are signed with K_{private} to form the *Signature* which is prepended to the data. Then, those four fields are encrypted with $K_{\text{subscription}}$ using the first 8 bytes of *Signature* as a nonce, which forms the final subscribe packet. This ensures that subscriptions can only be used by the decoder the subscription was generated for.

The decoder runs the same algorithm using the start and end timestamps as the encoder to figure out where the subtree root hashes are, so no extra metadata is required to positions the hashes, or to know how many there are.

This aspect of the design secures the system in the following ways:

- **AS2**: Pirated Subscription
  - Encrypting the subscription file with $K_{\text{subscription}}$ means that an attacker needs physical access to the decoder which a subscription was generated for. Additionally, signing the subscription with asymmetric cryptography ensures that further subscriptions are created by the encoder and cannot be forged by other decoders. In the Pirated Subscription scenario, the attacker is given a subscription file for a device id which they do not have, so it is not possible to extract the key and decrypt the subscription to apply it to the attacker's own decoder.

4.3. Signatures

The encoder uses the secret key SK to sign every message with a 64 byte Ed25519 signature. The public key PK is stored within all decoders as a static variable in the `.rodata` section and verifies every message to ensure it came from the encoder it was provisioned with.

This aspect of the design secures the system in the following ways:

- **SR2**: Authentication
  - Using asymmetric cryptographic, the decoder can ensure that it only accepts messages from the encoder it was provisioned with. Additionally, it prevents a compromised decoder from sending messages to other decoders, unlike symmetric cryptography.
- **AS5**: Pesky Neighbor
  - Bypassing the signature would require an attacker to write their own public key into the decoder which they want to receive messages on. In the Pesky Neighbor attack scenario, attackers do not have physical access to the neighbor's decoder, meaning that no side channel or other embedded attacks are possible. Additionally, using Zig, we can have memory safety guarantees which prevent memory vulnerabilities required to write to overwrite the neighbor's public key.

4.4. Global Timestamp

A global timestamp is kept in the RAM of the decoder, and is used to reject misordered frames. It increases on every successful decode.

This aspect of the design secures the system in the following ways:

- **SR3**: Increasing Timestamps
  - The timestamp can be checked against the timestamp in the frame before attempting to decode the data in frame, ensuring that the decoder never decodes an earlier frame.

4.5. Memory Protection Unit

4.6. Encryption at Rest

Before writing to flash, the contents of the page are first encrypted with a symmetric key K_{flash} using Salsa20. K_{flash} created by the decoder at build time and stored within the encrypted firmware (see Section 3.2). An incrementing nonce is stored next to each page in plaintext, to prevent nonce reuse. This adds another layer of security aiming to prevent offline attacks where a sophisticated attacker is able to dump the flash of the decoder and brute force the subscription's root hashes.

4.7. Zig

The Zig programming language was selected as the language to write the secure decoder in. Zig has memory safety features like protection from indexing an array out of bounds, double free protections and more. Additionally, it includes a comprehensive standard library which securely implements many cryptographic functions, among many other quality of life functions. Most importantly, Zig can do all this while maintaining perfect C compatibility and using the MSDK provided by Analog Devices Inc (the developer of the MAX78000FTHR), without requiring a rewrite of the Hardware Abstraction Layer (HAL). This allowed the team to move incredibly quickly while ensuring a high level of security.

5. Appendix

5.1. Proof that any interval can be encoded with at most 126 nodes

5.1.1. Definitions

A *node* is a tuple (o, l) where l is a power of 2 and o is a multiple of l. A node (o, l) *contains* a timestamp x if $x \in [o, o + l - 1]$. A *tree* is a finite set of nodes. To say that an interval I can be *encoded by* a tree T is to say that $\bigcup_{(o,l)\in T}[o, o + l - 1] = I$.

5.1.2. Lemma 1

For all $n \geq 1$, any interval $[0, k]$, where $k \leq 2^n - 1$, can be encoded by a tree containing at most n nodes.

Base case, $n = 1$:

We want to show that any interval $[0, k]$ where $k \leq 2^1 - 1 = 1$ can be encoded by at most 1 node.

- $\{(0, 1)\}$ encodes $[0, 0]$
- $\{(0, 2)\}$ encodes $[0, 1]$

Induction hypothesis: Suppose any interval $[0, k]$ where $k \leq 2^n - 1$ can be encoded by a tree with at most n nodes.

We wish to show that any interval on $[0, b]$ where $b \leq 2^{n+1} - 1$ can be encoded by a tree with at most $n + 1$ nodes.

Let $I = [0, b]$ where $b \leq 2^{n+1} - 1$.

Case 1: $b < 2^n$: By the induction hypothesis, I can be encoded with a tree with at most $n \leq n + 1$ nodes.

Case 2: $2^n \leq b < 2^{n+1}$: Then $I = [0, 2^n - 1] \cup [2^n, b]$.

The first interval, $[0, 2^n - 1]$ can be encoded by 1 node, namely, $(0, 2^n)$.

Since $b - 2^n < 2^n$, we can shift the second interval $[2^n, b]$ to the left by 2^n, obtaining the interval $[0, b - 2^n]$, which by the induction hypothesis can be encoded by a tree T containing at most n nodes.

Shifting that tree back to the right, we obtain $T' = \{(o + 2^n, l) \mid (o, l) \in T\}$ which encodes $[2^n, b]$. T' has the same number of nodes as T, so T' has at most n nodes.

Therefore, I is encoded by $\{(0, 2^n)\} \cup T'$, which contains at most $n + 1$ nodes.

5.1.3. Lemma 2

For all $n \geq 1$, any interval $[k, 2^n - 1]$ can be encoded with at most n nodes.

Let T be the tree which by Lemma 1 encodes $[0, 2^n - 1 - k]$ and contains at most n nodes. Then let $T' = \{(2^n - l - o, l) \mid (o, l) \in T\}$, which also contains at most n nodes. Intuitively, this is the tree T flipped horizontally.

We will show that T' encodes $[k, 2^n - 1]$ by considering an arbitrary timestamp $x \in [k, 2^n - 1]$. $2^n - 1 - x$ is within the interval $[0, k]$, so there exists a node $(o, l) \in T$ which contains $2^n - 1 - x$, that is, $o \leq 2^n - 1 - x \leq o + l$. Therefore, $2^n - 1 - l - o \leq x \leq (2^n - 1 - l - o) + l$, so the corresponding node in T', $(2^n - l - o, l)$, contains x.

5.1.4. Theorem

For all $n \geq 2$, any interval on $[0, 2^n - 1]$ can be encoded by at most $2n - 2$ nodes.

Base case, $n = 2$:

We want to show that all intervals on $[0, 3]$ can be encoded by at most 2 nodes.

- $\{(x, 1)\}$ encodes $[x, x]$ for $x \in [0, 3]$
- $\{(0, 2)\}$ encodes $[0, 1]$
- $\{(0, 2), (2, 1)\}$ encodes $[0, 2]$
- $\{(0, 4)\}$ encodes $[0, 3]$
- $\{(1, 1), (2, 1)\}$ encodes $[1, 2]$
- $\{(1, 1), (2, 2)\}$ encodes $[1, 3]$
- $\{(2, 1), (3, 1)\}$ encodes $[2, 3]$

Induction hypothesis: Suppose any interval on $[0, 2^n - 1]$ can be encoded by at most $2n - 2$ nodes.

We wish to show that any interval on $[0, 2^{n+1} - 1]$ can be encoded by at most $2(n + 1) - 2 = 2n$ nodes.

Let $I = [a, b]$ be a interval on $[0, 2^{n+1} - 1]$.

Case 1: $a \leq b < 2^n$. Intuitively, this means the range is within the left half of the tree. By the induction hypothesis, I can be encoded by $2n - 2 \leq 2n$ nodes.

Case 2: $2^n \leq a \leq b$. Intuitively, this means the range is within the right half of the tree.
The interval $[a - 2^n, b - 2^n]$ is a subinterval of $[0, 2^n - 1]$, so by the induction hypothesis, there is a tree T containing at most $2n - 2$ nodes which encodes $[a - 2^n, b - 2^n]$. Thus, the tree $T' = \{(o + 2^n, l) \mid (o, l) \in T\}$, also containing at most $2n - 2$ nodes, encodes $[a, b]$.

Case 3: $a < 2^n \leq b$. Intuitively, this means the interval crosses the center of the tree.
So $I = [a, 2^n - 1] \cup [2^n, b]$. By Lemma 2: 5.1.3, $[a, 2^n - 1]$ can be encoded by a tree T containing most n nodes. Similarly, by Lemma 1: 5.1.2, $[0, b - 2^n]$ can be encoded by can be encoded by a tree S containing at most n nodes, so $S' = \{(o + 2^n, l) \mid (o, l) \in S\}$ encodes $[2^n, b]$. Therefore, I can be encoded by $T \cup S'$, which contains at most $n + n = 2n$ nodes.

5.1.5. Application

Timestamps are 64 bit unsigned integers, so we choose $n = 64$. Theorem 5.1.4 shows that no more than $2(64) - 2 = 126$ nodes are required to encode any interval from $[0, 2^{64} - 1]$.

5.2. Proof of node partitioning optimality

We will show that the following algorithm correctly produces the optimal set of nodes to cover an interval $[a, b]$.

```python
class Node:
    offset: int
    power: int

def get_nodes(a: int, b: int) -> list[Node]:
    nodes = []

    while a <= b:
        power = min((b - a + 1).bit_length() - 1, ctz(a))
        ranges.append(Node(offset=a, power=power))
        a += 2 ** power

    return ranges
```

5.2.1. Correctness

5.2.2. Optimality

Bibliography

[1] MITRE, “2025 eCTF Documentation.” [Online]. Available: https://rules.ectf.mitre.org/2025